Efficient Heuristics for Classical Simulation of Quantum Circuits Using ZX-Calculus

Candidate no. TODO

Thesis word count: TODO

Submitted in partial completion of the
Master of Science in Advanced Computer Science

Trinity 2024

Abstract

Quantum circuits require validation and testing to ensure that they perform as expected. Classical simulation is a powerful tool for this purpose, but it is limited by the exponential growth of complexity with the number of qubits. Nevertheless, using the ZX-calculus, a graphical language for quantum mechanics, we can attempt to reduce how exponential this growth is. Recent developments have allowed focus to be placed on complexity being dependent on the number of T-gates that are in the circuit. Given t T-gates, if the complexity of the circuit is $O(2^{\alpha t})$, the aim is to reduce the value of α as much as possible.

Efficient Heuristics for Classical Simulation of Quantum Circuits Using ZX-Calculus

Candidate no. TODO

Word count: TODO

Submitted in partial completion of the

Master of Science in Advanced Computer Science

☐ I hereby certify that this is entirely my own work unless otherwise stated.

Trinity 2024

Contents

List of Figures

1 Introduction

Contents

1.1 Motivation

Quantum computing, a field rooted in the principles of quantum mechanics, represents a revolutionary approach to computation. Unlike classical computing, which relies on binary operations performed on bits, quantum computing utilizes quantum bits, or qubits, capable of representing and processing information in ways that classical bits cannot. This unique capability of qubits to exist in superpositions and entangle with each other allows quantum computers to perform certain types of calculations exponentially faster than their classical counterparts. If a classical computer were limited by memory and runtime, a quantum computer could potentially bypass these scaling issues - dubbed the "quantum advantage".

If scalable quantum computers were to become a reality, they could solve some of the most challenging problems in computer science much more efficiently than classical computers. One notable example is Shor's algorithm for factoring large numbers, which could break widely-used cryptographic systems such as RSA. Another significant algorithm is Grover's search algorithm, which provides quadratic speed-ups for unstructured search problems.

Classical computations play an essential role in assisting and accelerating the development of quantum computers. Classical simulation is a fundamental aspect of understanding and designing quantum hardware, often serving as the only means to validate these quantum systems [1]. Additionally, quantum algorithms can be implemented on classical simulators for testing and verification purposes while full-fledged quantum computers remain beyond reach (for now). These simulators emulate the behavior of quantum computers, allowing researchers to simulate processes as if running on actual quantum devices.

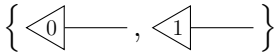
This thesis aims to present the speedup of classical simulation of quantum circuits in an almost entirely graphical approach. From the introduction of the qubits, to the representation of quantum circuits and linear maps, the underlying linear algebra that governs the operations of quantum computing is represented in a graphical manner. Using ZX-Calculus, a graphical calculus detailed in section 1.3, quantum circuits can be represented as diagrams, allowing for different discoveries to be more easily made when analysing the structure of said diagrams.

1.2 Basic Notation

In classical computing, a bit is the fundamental unit of information, and is denoted by a boolean value $b \in \{0, 1\}$. In quantum computing, the fundamental unit of information is the qubit, which we denote as $|b\rangle$, where $b \in \{0, 1\}$. Diagramatically, they are represented as follows:

We consider these the computational or Z-basis states.

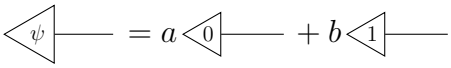
Definition 1.2.1 (Z-basis)**.** The *Z-basis* or computational basis is defined as



▲

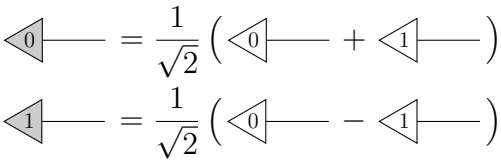
So far, the expressive power is still similar to classical computing. However, the power of quantum computing comes from the ability to exist in superpositions of these states. Concretely, the qubit can be represented in a linear combination of the basis states.

Definition 1.2.2 (Pure state)**.** A *pure state* $|\psi\rangle$ is a state



where $a, b \in \mathbb{C}$ and $|a|^2 + |b|^2 = 1$. ▲

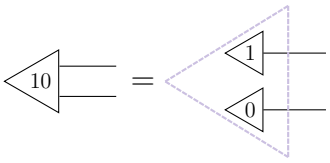
Then, the '+' or X-basis states can be defined.



(1.1)

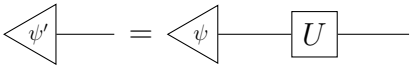
Along with the Z-basis states, these states form the building blocks of understanding quantum computing and the use of ZX-calculus for diagrammatic reasoning.

Intuitively, using the diagrammatic representation, we can consider combining diagrams in parallel. In the literature, this is the same as taking a *tensor product*. For example, we can have a 2-qubit state.



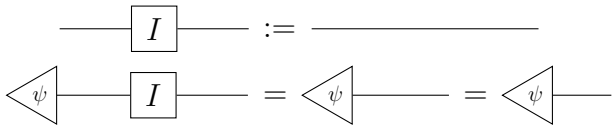
(1.2)

States can be transformed by quantum gates simply by *composing* them to obtain a new state. Given that composing the quantum gate U to a state $|\psi\rangle$ gives a state $|\psi'\rangle$, we can just connect the wires.



(1.3)

Intuitively, this means that the wire by itself should just be the identity gate, since composing it with any state should not change that state.

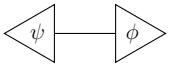


(1.4)

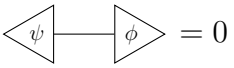
The dual of states are the effects. Here, we have the Z-basis effects.

For a state $|\psi\rangle$ and an effect $\langle\phi|$, we can obtain a scalar inner product. This is essentially the *bra-ket* notation in diagrammatic form by just connecting the wires, which comes with a few other definitions.

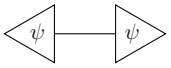
Definition 1.2.3 (Inner product [2])**.** The *inner product* $\langle\phi|\psi\rangle$ of two states $|\psi\rangle$ and $|\phi\rangle$ is defined as



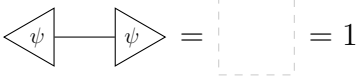
They are *orthogonal* if



The *squared-norm* of a state $|\psi\rangle$ is



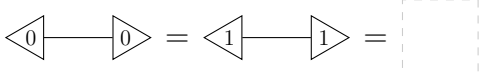
$|\psi\rangle$ is *normalised* if

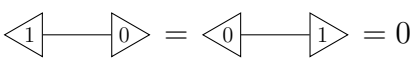


where the empty diagram represents the scalar 1. ▲

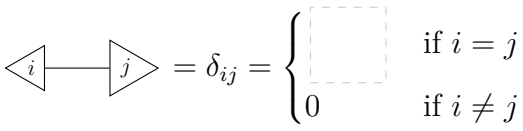
We can then define the inner product of the Z-basis states.

Definition 1.2.4. The inner product of the Z-basis states is





Succinctly, for $i, j \in \{0, 1\}$,

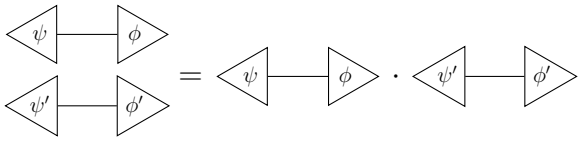


where δ_{ij} is the Kronecker delta. ▲

In fact, the succinct definition can be used to determine if a basis is an *orthonormal basis* (ONB). The X-basis is also an ONB.

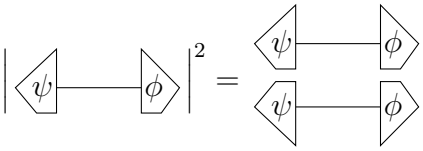
We are almost at the stage where we can obtain probabilities from the diagrams. The scalar inner product is not really a probability, that is, a number between 0 and 1, but instead, a number $z \in \mathbb{C}$.

First, we note that putting multiple scalars in a single diagram is just the same as multiplying them together.



(1.5)

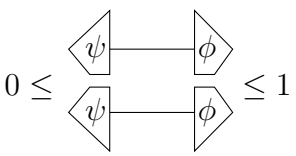
We also know that for a complex number and its conjugate, $z, \bar{z} \in \mathbb{C}, |z|^2 = z\bar{z}$ Translating that to the diagrammatic representation, we can obtain the following.



(1.6)

Note that assymetry is introduced to denote the conjugation. Remember that a complex number $z = a + bi \in \mathbb{C}$ has a conjugate $\bar{z} = a - bi \in \mathbb{C}$, so this corresponds to a vertical reflection in the diagram above.

Using the *Born rule*, if we make sure that $|\psi\rangle$ and $|\phi\rangle$ are normalised, we can obtain the following.



(1.7)

This can be interpreted as our desired probaility - the probability of measuring $|\psi\rangle$ in the state $|\phi\rangle$.

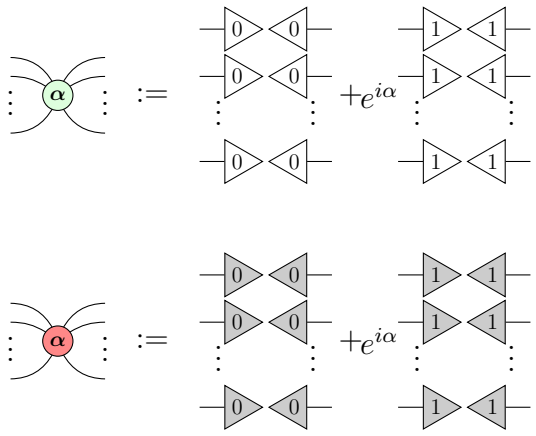
We now have all we need to introduce the ZX-calculus notation.

1.3 ZX-Calculus

So far, we have introduced the representation of quantum states, effects, and inner products in a diagrammatic form. However, there still isn't much we can do without a way to manipulate them. In the previous section, we mentioned that we can apply quantum gates to states to obtain new states. Here, we will introduce the ZX-Calculus and show the highly intuitive representation of some quantum gates such as the Hadamard and CNOT gates, based on the building blocks from the previous section. The ZX-Calculus is a graphical calculus for quantum computations, developed by Coecke and Duncan in 2008 [3]. All operations are expressed as *spiders*, connected by *edges* (or *wires*).

1.3.1 Spiders

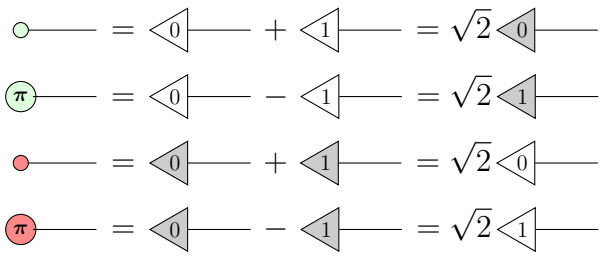
Definition 1.3.1 (Spiders)**.**



▲

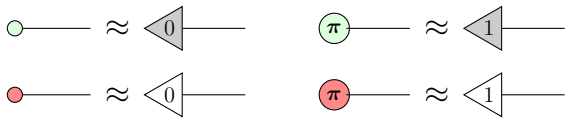
Here, we have green and red spiders, also known as Z and X spiders, made of the Z and X basis states respectively. Wires coming in from the left are *inputs*, and wires going out to the right are *outputs*. The α inside a spider node is also called the *phase* of the spider. For example, with $\alpha = 0, e^{i\alpha} =$. It is convenient to denote spiders with $\alpha = 0$ as having no angle written inside the spider.

Immediately, we can see that the Z and X basis states can be formed from the definition of the spiders, up to the multiplicative constant $\sqrt{2}$. Note that $e^{i\pi} = -1$.



(1.8)

It turns out that in reasoning with spiders, we can ignore the multiplicative constants, since much of the manipulation of the diagrams is based on the structure of the diagrams themselves. The scalars are easy to determine using the definitions we have seen so far. By using $\approx$ to denote equality up to a non-zero factor, we have

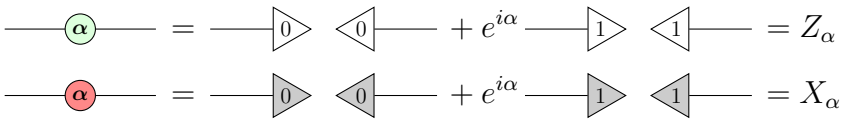


(1.9)

Scalars can also be represented using just spiders.

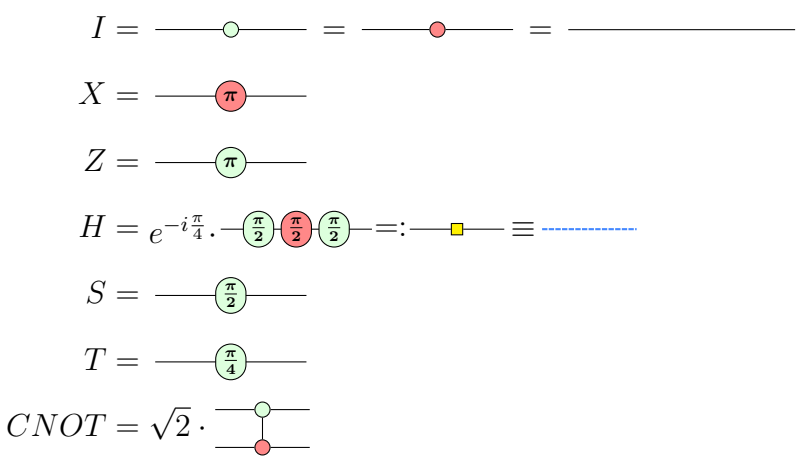
(1.10)

We can also see the single-qubit gates in the ZX-Calculus.



(1.11)

We can then define many of the common quantum gates in this way.

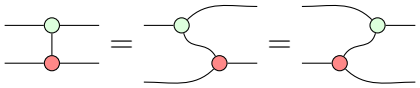


(1.12)

There are 2 gates of special note here. The Hadamard gate H, denoted by the small yellow box, or as a blue dashed line, is a single-qubit gate that interchanges

Z and X bases. We will see later in the ZX-rules that this is vital for the colour change (***cc***) rule, as well as why it is useful to denote it as the blue dashed line.

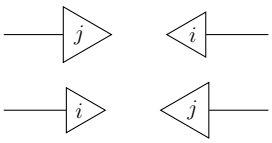
The CNOT gate is a two-qubit gate that acts on the target qubit if the control qubit is in the state $|1\rangle \approx$. The vertical line is used simply because it does not matter which direction the line is going, where the following equality of the second and third forms can be verified.



(1.13)

Last but not least, we also introduce the SWAP gate, which as the name suggests, swaps two qubits.

$$SWAP = \quad := \sum_{i,j\in\{0,1\}}$$



(1.14)

We can now introduce the ZX-rules in the next section.

1.3.2 ZX-Rules

The ZX-Calculus is based on a set of rules that allow us to manipulate the diagrams. For $\alpha, \beta \in [0, 2\pi)$ and $a \in \{0, 1\}$, the rules are as follows.

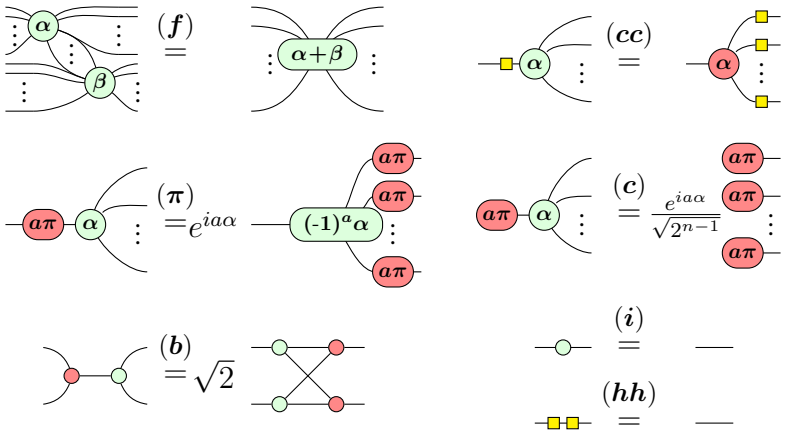


Figure 1.1: *ZX*-rules presented in [4]

The names of the rules are spider (***f***)usion, colour change (***cc***) rule, (***π***)-commutation, (***c***)opy rule, (***b***)ialgebra, (***i***)dentity rule, and (***hh***)adamard cancel

rule. For (***c***), n refers to the number of outputs. By duality, the rules also hold if the colours are swapped.

The power of ZX-diagrams comes from the fact that two diagrams are equivalent as long as the connectivity of the diagram is the same - that is, as long as order of inputs and outputs of the whole diagram is preserved. This does not depend on the position of the spiders or the direction of wires between them. This neat feature of ZX-diagrams is called "Only Connectivity Matters" (OCM).

We can then obtain caps and cups using 2-legged spiders together with (***i***).

(1.15)

Caps and cups obey the *yanking equations*.

(1.16)

Before we move on to reasoning about the diagrams in a graph-theoretic manner, we first introduce the concept of a *Clifford+T diagram*.

Definition 1.3.2 (Clifford diagram)**.** A *Clifford diagram* is a ZX-diagram where the phases are restricted to integer multiples of $\frac{\pi}{2}$. ▲

Definition 1.3.3 (Clifford+T diagram)**.** A *Clifford+T diagram* is a ZX-diagram where the phases are restricted to integer multiples of $\frac{\pi}{4}$. ▲

It is clear from the definitions that a Clifford diagram is a subset of a Clifford+T diagram. The reason we introduce these concepts is that in trying to obtain our main goal of the thesis, it is sufficient to reason about Clifford+T diagrams, as a result of the following theorem.

Theorem 1.3.4 (Approximate Universality [5])**.** Clifford+T diagrams are *approximately universal* for quantum computing - they can approximate any quantum circuit.

The proof of the above theorem involves the *Solovay-Kitaev theorem*, combining with the fact that any single-qubit phase gate can be approximated by a sequence of Hadamard and T gates, with details found in [5].

1.4 Graph Reduction

Because of OCM, ZX-diagrams up until now have very similar structure to graphs, where the spiders are the vertices and the wires are the edges. The only difference may be that the spiders are coloured, and that the Hadamard boxes are included. In this section, we solidify the connection between ZX-diagrams and graphs, and introduce an algorithm that will simplify ZX-diagrams, defining what it means for a graph to be simplified. Ultimately, simplifying ZX-diagrams should allow us to reduce the number of spiders in the diagram, which directly translates to a reduction in the number of gates in the quantum circuit. In turn, as we will see in the next chapter, this will lead to a speedup in the simulation of quantum circuits.

First, we define what it means for ZX-diagram to be *graph-like*.

Definition 1.4.1 (Graph-like [6])**.** A ZX-diagram is *graph-like* when

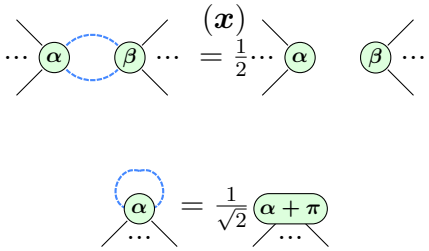
- Every spider is a Z-spider.
- Spiders are only connected via Hadamard edges.
- There is at most only one edge between each pair of spiders.
- There are no self-loops.
- Every input and output wire is connected to a Z-spider.

▲

Just from this definition, Hadamard edges are in fact the only edges in the graph-like ZX-diagram, leading to the convenience of having them as the blue dashed lines introduced earlier in (1.12). The following proposition can then be proven, using rewrite rules introduced previously in ZX-Rules.

Proposition 1.4.2 ([6])**.** Every ZX-diagram can be converted to a graph-like ZX-diagram in polynomial time.

The proof [6, 7] makes use of the fact that Hadamard edges in parallel cancel, and that we can always remove a Hadamard self-loop. Parallel edges cancelling is an application of the *Hopf rule* ($\boldsymbol{x}$), derivable from the ZX-rules [1].



(1.17)

We can simplify the graph-like ZX-diagram further. Now, the following two rewrite rules, derivable from the original ZX-rules, are introduced.

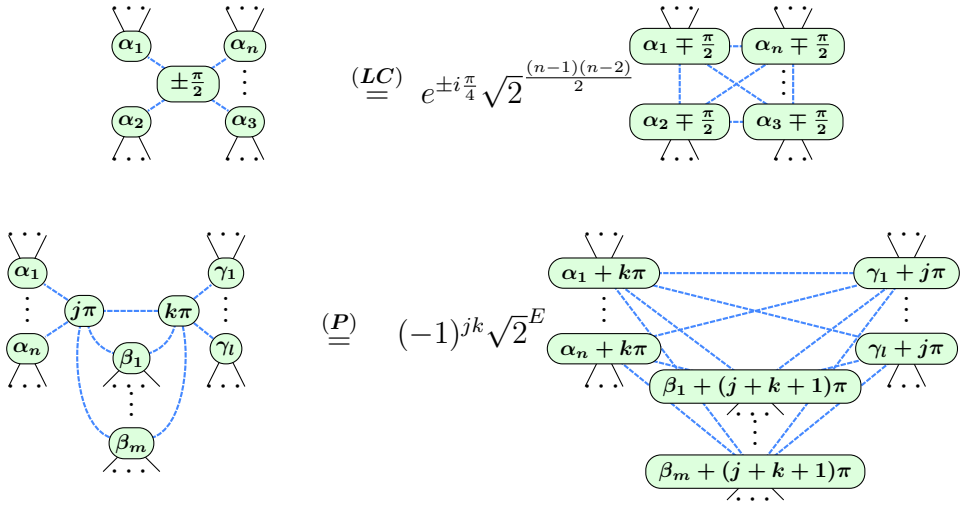


Figure 1.2: Derived ZX-rules [7], where $\alpha, \beta, \gamma \in [0, 2\pi)$ and $j, k \in \{0, 1\}$.

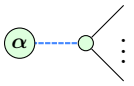
The rules are *local complementation* ($\boldsymbol{LC}$) and *pivoting* ($\boldsymbol{P}$) respectively. Here, $E = (n-1)m + (l-1)m + (n-1)(l-1)$.

Repeatedly simplifying and reducing diagrams in this manner will end up giving us patterns in the diagrams. The following definitions help us identify these patterns. First, we define interior and boundary spiders, and then phase gadgets.

[1] In fact, ($\boldsymbol{b}$) $\implies$ ($\boldsymbol{x}$)

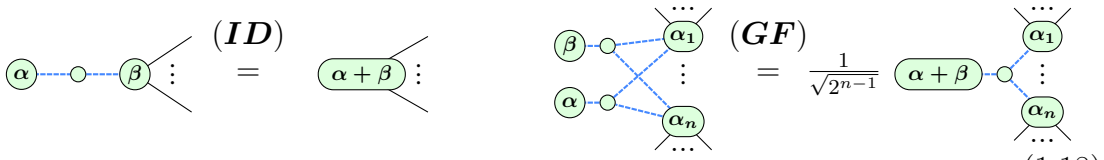
Definition 1.4.3 ([8])**.** A *boundary spider* is a spider connected to an input or output. All other spiders are *internal spiders*. ▲

Definition 1.4.4 (Phase gadget [8])**.** A *phase gadget* is an arity-1 spider with angle α, connected via a Hadamard edge to a spider with no angle.



▲

The usefulness of phase gadgets is that they can be used to simplify the diagram further. Here, two more derived rules involve phase gadgets.



(1.18)

These help us define the reduced gadget form.

Definition 1.4.5 (Reduced gadget form [8])**.** A graph-like ZX-diagram is in *reduced gadget form* when

- Every internal spider is a non-Clifford spider or part of a non-Clifford phase-gadget.
- Every phase-gadget has more than one target.
- No two phase-gadgets have the same set of targets.

▲

It turns out that there is an algorithm detailed below that is bounded in $O(n^3)$ where n is the number of spiders, to convert any graph-like ZX-diagram to reduced gadget form [9]. Thus, much of the work in this thesis will focus on the reduced gadget form, as it tends to bring us closer to the goal of reducing overall T-count. However, there are exceptions to this, as we will see in the next chapter.

Algorithm 1.4.6 (ZX-simplify [9])**.** Starting with a graph-like ZX-diagram, do the following:

1. Apply (***LC***) until all proper Clifford spiders are removed.
2. Apply (***P***) (and variations) until all interior $k\pi$-spiders are removed or transformed into phase-gadgets.
3. Remove all Clifford phase-gadgets using (***LC***) and (***P***).
4. Apply (***ID***) and (***GF***) wherever possible. If any matches are found, go back to step 1. Otherwise, the diagram is in reduced gadget form.

Example 1.4.7. The following diagram from [9] is in reduced gadget form. ▲

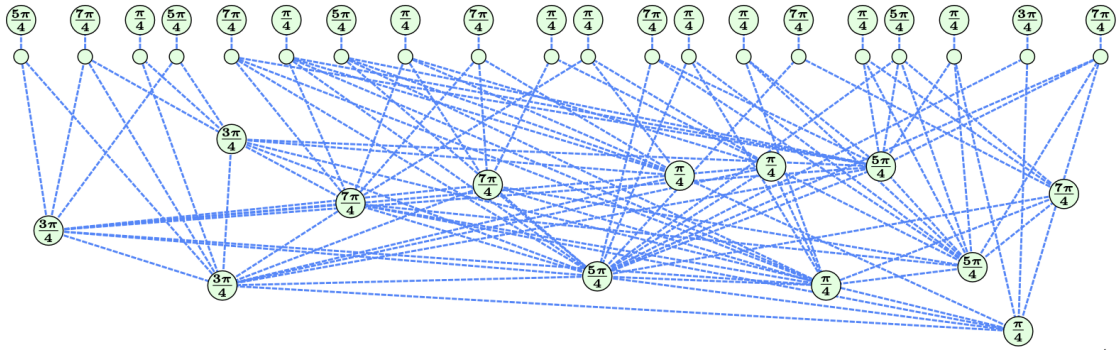


Figure 1.3: A ZX-diagram in reduced gadget form [9]

2
Classical Simulation

Contents

2.1 Introduction

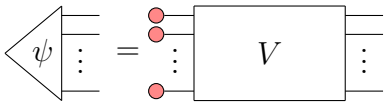
As mentioned in the previous chapter, classical simulation of quantum circuits provides an essential tool for verification and validation, despite its inherent limitations. This chapter delves into the methods and advancements in classical simulation via stabiliser decomposition, discussing the role of magic states, cat states, and graph cuts, in extending the capabilities of these simulations. Classical simulation methods that store a complete description of an n-qubit quantum state as a complex vector of size 2^n are effective only for a small number of qubits, typically around 30. For example, Shor's factoring algorithm has been simulated with 31 qubits and approximately half a million gates [10]. However, for certain classes of quantum circuits, such as those that can be represented in a Clifford

diagram, classical simulation can be significantly more efficient [11]. This is known as the *Gottesman-Knill theorem*.

Theorem 2.1.1 (Gottesman-Knill Theorem)**.** A Clifford computation can be efficiently classically simulated.

In other words, a computation involving only *Clifford diagrams* can be efficiently classically simulated. This is extremely useful when considering that ZX-Calculus simplifies the process of identifying and manipulating *stabiliser states*.

Definition 2.1.2 (Stabiliser state [11, 12])**.** An n-qubit state $|\psi\rangle$ is a *stabiliser state* if it has the form

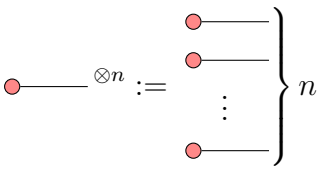


where V is a Clifford diagram. ▲

It is clear that if a computation only involves stabiliser states, then it can be efficiently classically simulated. Stabiliser circuits are particularly notable for their applications in quantum error correction [11]. In the next few sections, we will see more clearly how these stabiliser states are used in classical simulation. Particularly, we also need to consider Clifford+T diagrams, and how we can still (inefficiently) simulate them using stabiliser states.

2.2 Strong Simulation

When simulating quantum circuits, we assume that the input state to our n-qubit circuits is of the form



(2.1)

Given that we have a n-qubit circuit with the input state from (2.1), we would then want to obtain the probability of measuring a certain output state. For instance, we can plug in the outputs $\langle \vec{x}|$ for $\vec{x} = x_1, x_2, ..., x_n$ as spiders as in Figure 2.1.

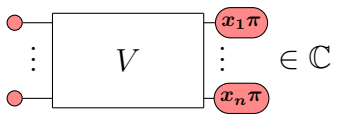
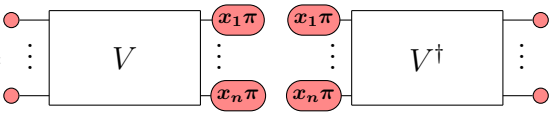


Figure 2.1: Scalar involving Clifford+T diagram V

Then using the Born rule, we can obtain the probability of measuring the output state [6].

$$\Pr(x_1, x_2, ..., x_n) = \frac{1}{\sqrt{2^{4n}}}$$ 

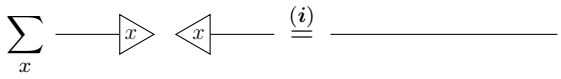
(2.2)

$V^\dagger$ is used to denote the adjoint of V, which simply means the inputs and outputs are swapped, and the spider phases are negated. The scalar factors of $1/\sqrt{2}$ comes from those seen in (1.8). If V is a Clifford diagram, then following from the Gottesman-Knill Theorem, the scalars can be computed efficiently. Obtaining the probabilities this way is known as *strong simulation*.

Marginal probabilities can also be obtained similarly. Recall that using the law of total probability, we have for $k < n$,

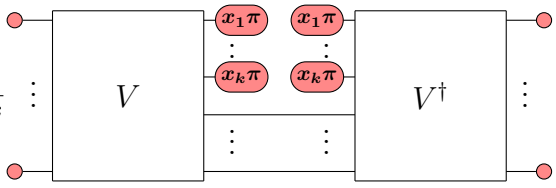
$$\Pr(x_1, x_2, ..., x_k) = \sum_{x_{k+1}, x_{k+2}, ..., x_n} \Pr(x_1, x_2, ..., x_k, x_{k+1}, ..., x_n) \tag{2.3}$$

Then, applying the (2.3) to (2.2), and using the ($\boldsymbol{i}$)dentity rule, where



(2.4)

we can obtain the marginal probabilities.

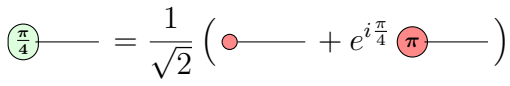
$$\Pr(x_1, x_2, ..., x_k) = \frac{1}{2^{n+k}}$$ 

(2.5)

A natural question that arises is whether the same can be done if V is a Clifford+T diagram instead. The following sections will delve into the methods used to simulate such circuits.

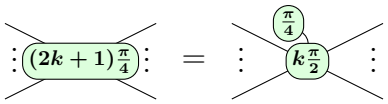
2.3 Stabiliser Decomposition

Definition 2.3.1 (Magic state [12])**.** The *magic state* is



▲

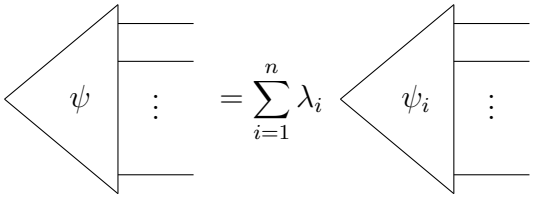
The naïve approach to make use of this magic state decomposition is to then apply it to every single T-like spider in the Clifford+T diagram, where we define a T-like spider as a spider with a phase of $(2k+1)\frac{\pi}{4}$ for some $k \in \mathbb{Z}$. By un($\boldsymbol{f}$)using the magic state from these spiders as below, we can apply the Magic state [12] decomposition to obtain a sum of stabiliser states.



(2.6)

This sum can be called the *stabiliser decomposition* of the circuit.

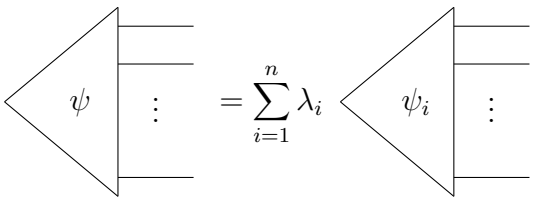
Definition 2.3.2 (Stabiliser Decomposition [12])**.** The *stabiliser decomposition* of a state $|\psi\rangle$ is a sum of stabiliser states



where $\lambda_i \in \mathbb{C}$ and $|\psi_i\rangle$ are stabiliser states. ▲

More generally, a decomposition need not consist only of stabiliser states.

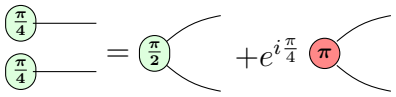
Definition 2.3.3 (Decomposition)**.** The *decomposition* of a state $|\psi\rangle$ is a sum of states



where $\lambda_i \in \mathbb{C}$ and $|\psi_i\rangle$ are states. For any decomposition d, define its length to be $|d| = n$. ▲

Using the Magic state [12] decomposition, we can see that the value of $n = 2^t$ if we apply it to the t T-like spiders in the circuit. We can call t the *T-count* of the circuit.

However, there is an even better decomposition



(2.7)

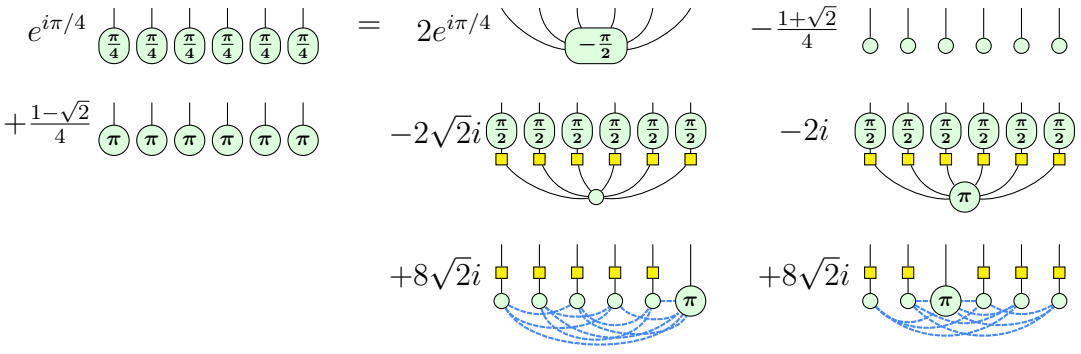
This allows us to decompose the circuit into a sum of stabiliser states with $n = 2^{\frac{t}{2}}$. Different methods of decomposing the circuit can lead to different values of n. Since n is believed to be exponential in t (or else we have $P = NP$) [13], we can take it that $n = O(2^{\alpha t})$ for some $0 < \alpha < 1$. We can then compare the efficiency of different methods by using this metric.

Definition 2.3.4 ((In)efficiency [9, 12, 14, 15])**.** The *(in)efficiency* of a decomposition D that reduces the T-count by r using p states is

$$\alpha(D) := \frac{\log_2(p)}{r} \tag{2.8}$$

where a lower value of α [1] indicates a more efficient decomposition. ▲

For example, the Bravyi-Smith-Smolin (BSS) decomposition [12] makes use of the following decomposition which has $\alpha \approx 0.468$.



(2.9)

Using this metric, we can even compare the efficiency of different algorithms used to decompose the circuit. It also allows us to obtain the efficiency of different decompositions which do not use magic states at all. As we will see in the

[1] We omit D when the context is clear

next few chapters, this ties in well when we consider the higher efficiency of *cat state* decomposition.

A closely related metric we can use is the *stabiliser rank*.

Definition 2.3.5. Let D be the set of all stabiliser decompositions of a state $|\psi\rangle$. The *stabiliser rank* $\chi(|\psi\rangle)$ of a state $|\psi\rangle$ is

$$\chi(|\psi\rangle) := \min\{n \mid \exists(d_1, d_2, ..., d_n) \in D \text{ s.t. } |\psi\rangle = \sum_{i=1}^{n} d_i\}$$

▲

We can see that the relation between the stabiliser rank and its efficiency is

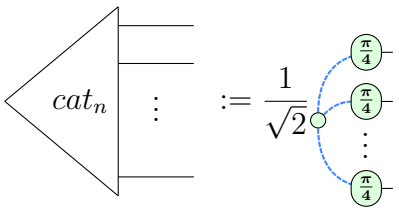
$$\chi(|\psi\rangle) = 2^{\alpha_\chi(|\psi\rangle)t} \tag{2.10}$$

with the T-count t of the circuit.

The goal for any stabiliser decomposition is to obtain an efficiency as close to α_χ as possible.

2.4 Cat States

Definition 2.4.1 (Cat state [16])**.** A *cat state* is defined as

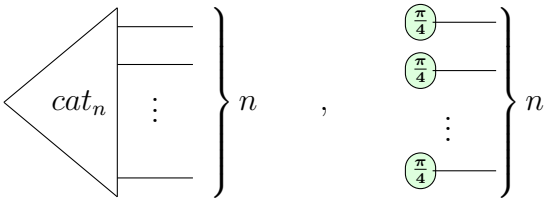


(2.11)

▲

Cat states are used because their decompositions have low value of α, as well as the following proposition, which bounds the α of the corresponding magic state decomposition.

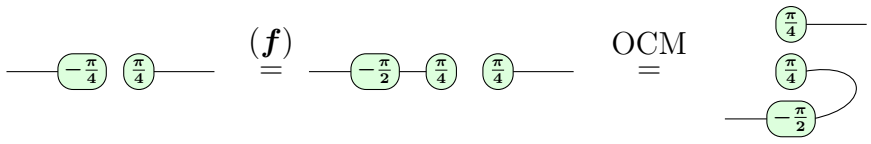
Proposition 2.4.2. Suppose α_1, α_2 are the efficiency metrics of the decompositions of



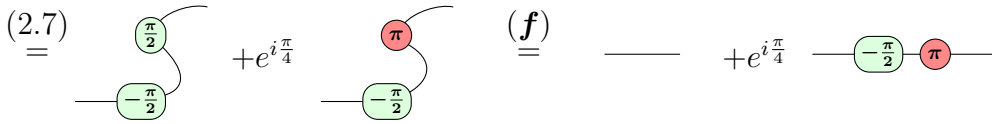
respectively. Then

$$\alpha_1 \leq r \implies \alpha_2 \leq r + \frac{1}{n}$$

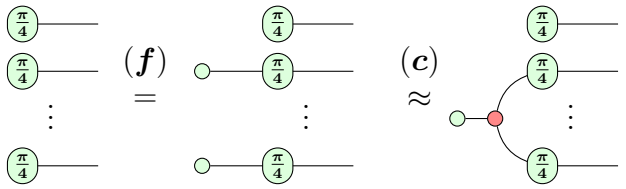
Proof. The proof is adapted from [16]. Suppose $\alpha_1 \leq r$. First, we show the following equality.



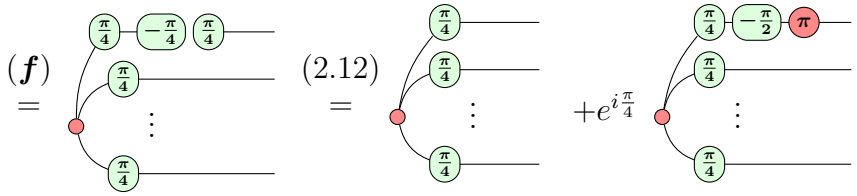
(2.12)



Then we can show the following.



(2.13)

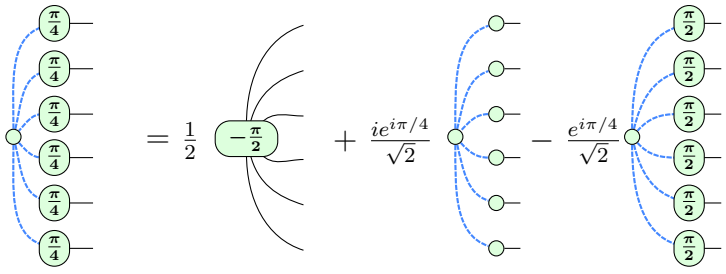


There are 2 copies of $|\text{cat}_n\rangle$ which would each give at most $2^{n\alpha_1}$ stabiliser states, for a total of at most $2^{n\alpha_1+1}$ stabiliser states. Thus, we have

$$\alpha_2 \leq \frac{n\alpha_1 + 1}{n} \leq r + \frac{1}{n}$$

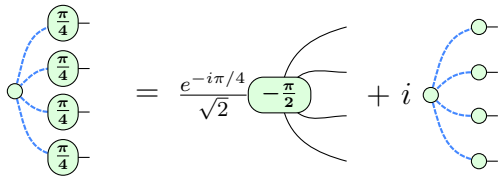
□

There are various cat state decompositions that have been found. For example, the decomposition for $|\text{cat}_6\rangle$ is



(2.14)

It has an efficiency of $\alpha \approx 0.264$. Even better, for $|\text{cat}_4\rangle$, we have the decomposition



(2.15)

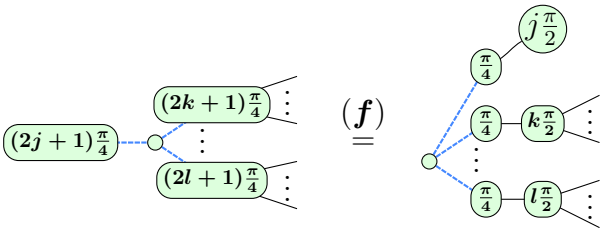
which has an efficiency of $\alpha = 0.25$.

Table 2.1 shows the best known efficiencies of the $|\text{cat}_n\rangle$ decomposition for different values of n [14].

n	3	4	5	6	7	8	9	10	11	12	13	14
terms	2	2	3	3	6	6	9	9	27	27	27	27
$\sim \alpha$	0.333	0.25	0.317	0.264	0.369	0.323	0.352	0.317	0.432	0.396	0.366	0.340

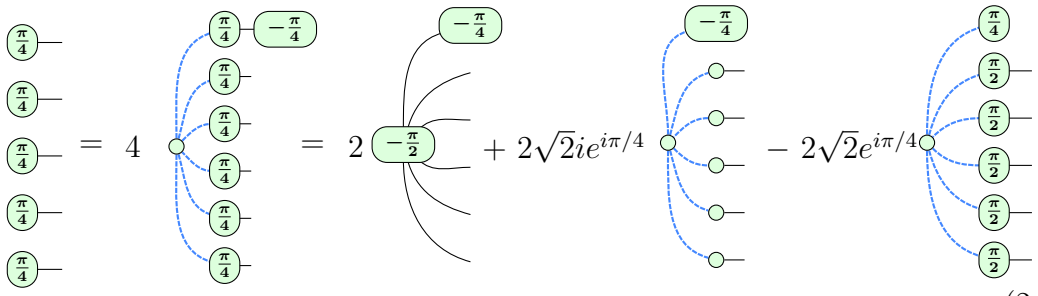
Table 2.1: $|\text{cat}_n\rangle$ decomposition

Recall that if we are dealing with the Reduced gadget form [8], any internal spider that is Clifford will be part of a non-Clifford phase gadget, by definition. Further, no two Clifford spiders will be neighbours [15]. Then, we can apply the cat state decompositions to the cat states centered at the 0-spiders that are part of these phase gadgets in the reduced gadget form, if they have ≥ 3 neighbours. This can be seen below, where $j, k, l \in \mathbb{Z}$.



(2.16)

However, if the cat states are not present, we would need to fallback to a magic state decomposition. Here, the discovery of the (non-stabiliser) decomposition of 5-qubit magic states using the decomposition of $|\text{cat}_6\rangle$ gives us a guaranteed way to decompose a circuit with a T-count ≥ 5 [15].



(2.17)

We then have a resulting $\alpha = \frac{\log_2(3)}{4} \approx 0.396$, which is already better than the BSS decomposition. In the next chapter, we will continue using the algorithm from [15] and improve upon the use of these cat and magic state decompositions.

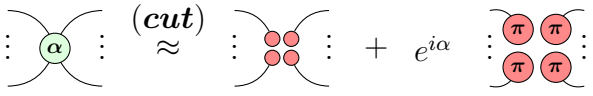
2.5 Graph Cuts

In a complementary approach to reducing the effective α of the stabiliser decomposition, graph cuts offer a different perspective [17]. Suppose we have 2 Clifford+T diagrams S_1, S_2 of T-counts t_1, t_2 that both have a stabiliser decomposition efficiency of α.

(2.18)

If we combine the diagrams into a single diagram S, we could expect the stabiliser decomposition efficiency of S to be α, coming from having $2^{\alpha(t_1+t_2)}$ stabiliser states. However, since we can treat S_1, S_2 independently and multiply their stabiliser decompositions, the actual number of stabiliser states is instead $2^{\alpha t_1} + 2^{\alpha t_2} \leq 2^{\alpha t+1}$, where $t = \max(t_1, t_2)$. This gives a resulting $\alpha' \leq \alpha \frac{t+1}{t_1+t_2}$. In the ideal case, we have that $t_1 = t_2$, so $\alpha' \leq \frac{\alpha}{2} + \frac{1}{2t}$, a vast improvement especially for a high T-count. It is with this motivation that graph cuts are another promising approach.

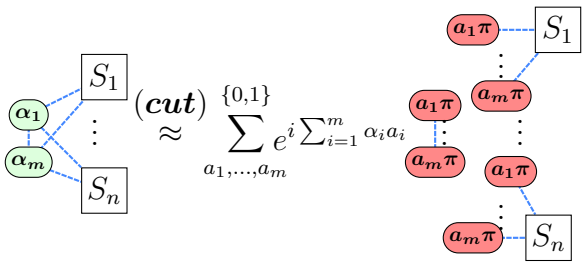
First, by definition, we can see that



(2.19)

Note that while this decomposition (***cut***) is not necessarily on a state, because of OCM, and the use of cups and caps, we can easily consider this to fit the definition of a Decomposition.

Then if we have a diagram with subdiagrams $S_1, ..., S_n$, we can see that the following holds.



(2.20)

At the cost of m cuts, and thus 2^m terms, the diagram is split into $\leq m^2 + n$ subdiagrams that can be treated independently. The $\leq m^2$ scalars in the form of spider-pairs are trivially scalars (1.10), and there are only $\leq n$ subdiagrams with T-counts > 0. If each subdiagram S_i has the T-count of t_i, we have a sum of 2^m terms, each value which can be obtained with just $\leq m^2 + \sum_{i=1}^n 2^{\alpha t_i}$ stabilizer terms. This gives a total of $\leq 2^m(m^2 + \sum_{i=1}^n 2^{\alpha t_i})$ stabilizer terms. We can obtain the efficiency.

$$\begin{aligned} \alpha' &\leq \frac{\log_2(2^m(m^2 + \prod_{i=1}^n 2^{\alpha t_i}))}{\sum_{i=1}^n t_i} \\ &= \frac{m + \log_2(m^2 + \prod_{i=1}^n 2^{\alpha t_i})}{\sum_{i=1}^n t_i} \\ &\leq \frac{m + 2\log_2(m) + \sum_{i=1}^n \alpha t_i}{\sum_{i=1}^n t_i} \end{aligned} \tag{2.21}$$

In the ideal case where $t_i = t$ for all i, we can produce a crude upper bound

$$\begin{aligned} \alpha' &\leq \frac{\alpha}{n} + \frac{m + \log_2(m^2 n)}{nt} \\ &< \frac{\alpha}{n} + \frac{3m + n}{nt} \\ &= \frac{\alpha}{n} + \frac{3m}{nt} + \frac{1}{t} \end{aligned} \tag{2.22}$$

Keeping the ratio m/n small is key to reducing the effective α. Clearly, fewer cuts are better, with preferably the end result being many disconnected subdiagrams.

Another way graph cuts also reduce the effective α need not necessarily be by obtaining disconnected subdiagrams. On pseudo-structured circuits produced

from CNOTs, phase gates, and Toffolis, graph cuts were used to take advantage of the structure to optimise for the use of spider (***f***)usion in cascading fashion, where a single cut can lead to a large reduction in the T-count [18]. There, the effective efficiency obtained was $0.1 \lesssim \alpha \lesssim 0.2$. More details on this will be discussed in section 3.3.

The same was also found for extremely structured circuits involving cat states, where a single decomposition reduced the T-count by 286, for an $\alpha \approx 0.0035$ [17], though this was an ideal case which is unlikely to occur in practice.

2.6 Subgraph Complement

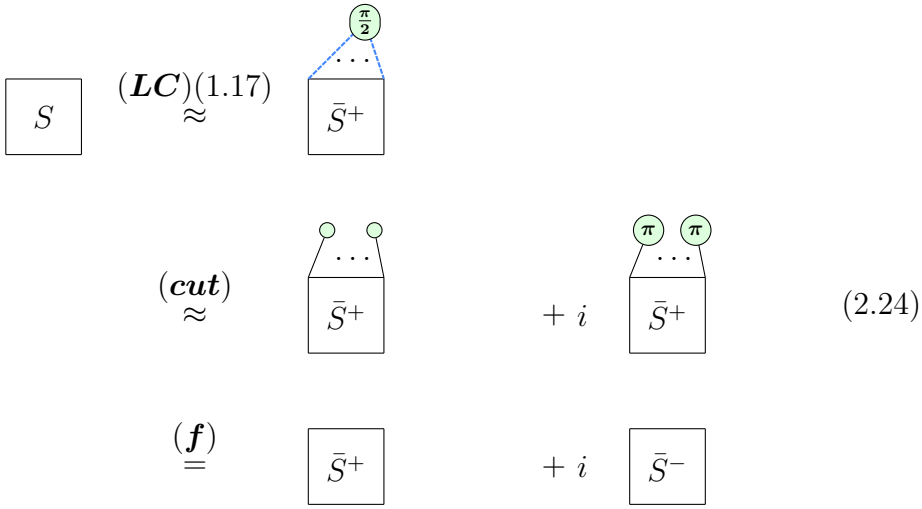
A further addition to the strength of graph cuts is the subgraph complement approach. For a graph-like ZX-diagram (V, E), with K being the set of edges for a complete graph on V, its *complement* is simply $(V, K \backslash E)$. We define a *subdiagram* to simply be a subgraph of the graph-like ZX-diagram. If we analyse (***LC***) closely, and consider the Hopf rule (***x***) making parallel edges cancel, there seems to be a way to obtain a subgraph complement with some manipulation of ZX-rules. In fact, we have the following result.

Proposition 2.6.1 ([17])**.** Suppose we have a subdiagram S of a graph-like ZX-diagram. Then

$$\boxed{S} \approx \boxed{\bar{S}^+} + i\boxed{\bar{S}^-} \tag{2.23}$$

where $\bar{S}^+, \bar{S}^-$ are the complements of S where the spider phases differ from S by $\pm\frac{\pi}{2}$ respectively.

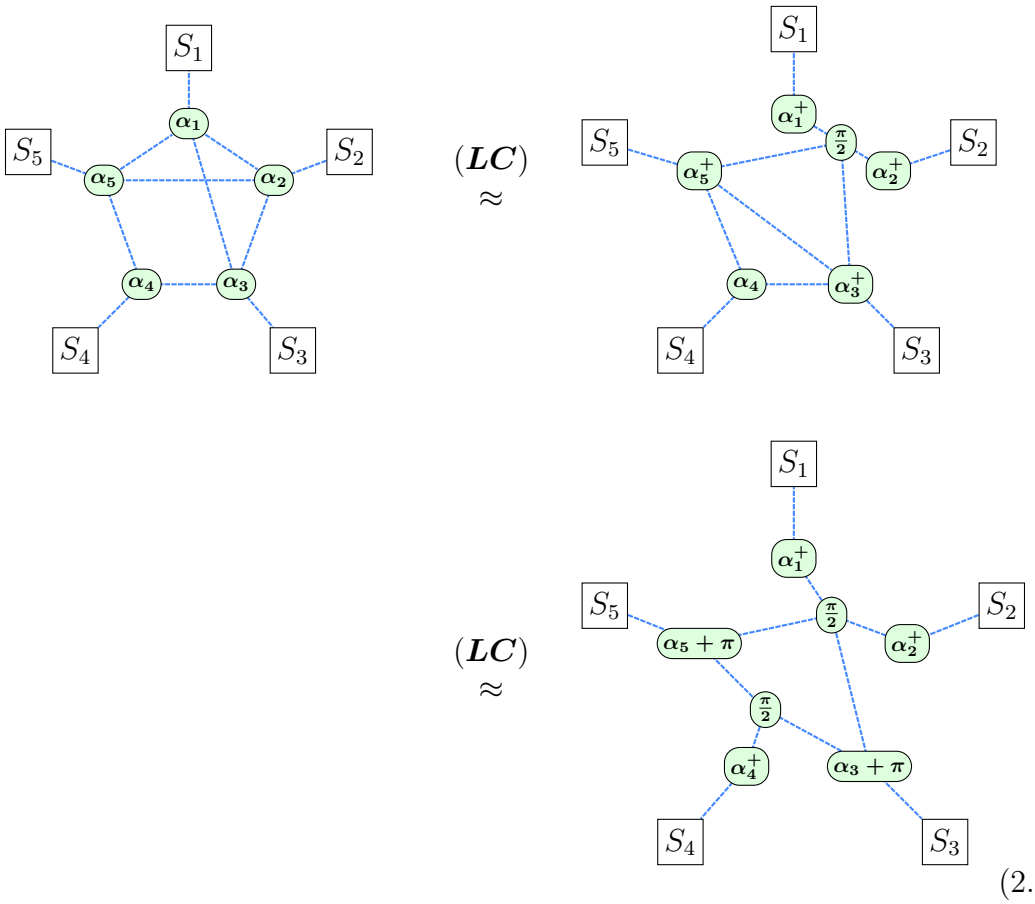
Proof. The proof is adapted from [17].



(2.24)

□

The power of this proposition can be demonstrated with the following example. Suppose we have the following graph-like ZX-diagram, where S_i are subdiagrams.



(2.25)

Following from (2.20), we can apply 5 cuts to split the diagram into 5 disconnected subdiagrams, giving us 2^5 terms. However, we can apply Prop. 2.6.1 twice to give us 2^2 terms instead.

3
Search

Contents

3.1 Introduction

The task of obtaining the stabiliser decomposition for classical simulation purposes has only been discussed at a high level so far. While we have the various methods of magic state, cat state decompositions, along with grap cut decompositions, we have not yet detailed the specific steps taken to obtain the stabiliser decompositions in the most efficient way so far.

In modeling our task, we can think of the stabiliser decomposition as a search problem, where we are searching for the most efficient stabiliser decomposition. We can visualize an example as in Figure 2.1, where V is a Clifford+T diagram at the

root. The directed edges represent the decomposition used from the parent to the child node, and the leaves d_i are the possible stabiliser decompositions.

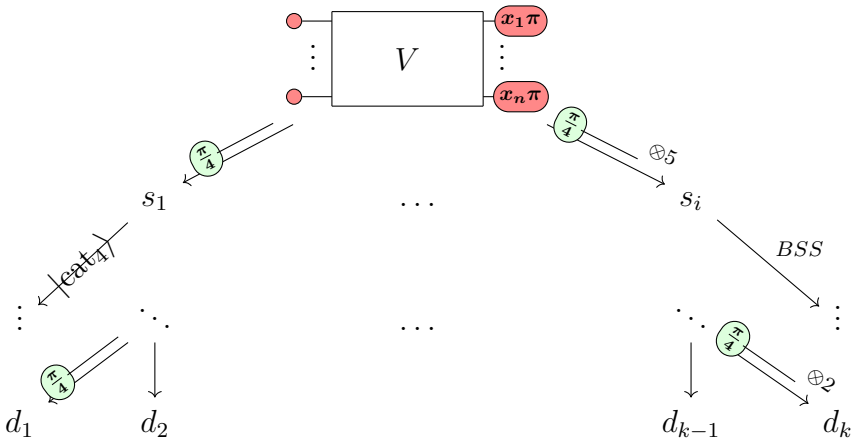


Figure 3.1: Example search tree for a stabiliser decomposition

---

To traverse the search tree, we might take the following steps [9, 15].

1. Perform necessary spider (***f***)usions to reduce to graph-like form.
2. Apply a chosen decomposition ("choose an edge").
3. Simplify the resulting Clifford+T diagram using ZX-simplify. Some Clifford spiders may be removed, and some T-like spiders may be combined.
4. Repeat steps 1-3 until the T-count is 0.

The complexity from using this algorithm was found to be $O(N^3 + 2^{\alpha t}t^2)$, where N is the number of gates in the original circuit [9]. Clearly, any circuit with a T-count that is not insignificant will mean that the complexity is dominated by $O(2^{\alpha t}t^2)$.

In this thesis, we will focus on the second step of the algorithm, where we choose the decomposition to apply at each node. There are a variety of ways to choose a decomposition, detailed in the next few sections.

3.2 Greedy Algorithm

The most obvious way to obtain an efficient decomposition is to, at each step, use the decomposition that has the smallest associated value of α as possible. This is

essentially a greedy algorithm, and is the method used in [9, 15]. Intuitively, this should give us a decomposition with an α upper bounded by the worst decomposition available. More formally, we have the following lemma.

Lemma 3.2.1. Let d be a stabiliser decomposition, and let $\alpha_1, ..., \alpha_n$ and $t_1, ..., t_n$ be the α values and T-count reductions of each of the n decompositions to obtain d from the initial Clifford+T diagram. Then the efficiency α value of the decomposition d is given by

$$\alpha(d) \leq \frac{\sum_{i=1}^{n} \alpha_i t_i}{\sum_{i=1}^{n} t_i} \tag{3.1}$$

Proof. Each decomposition gives $\leq 2^{\alpha_i t_i}$ terms, for a total of

$$\leq \prod_{i=1}^{n} 2^{\alpha_i t_i} = 2^{\sum_{i=1}^{n} \alpha_i t_i}$$

terms. Since the final T-count is 0, the initial T-count is $\sum_{i=1}^{n} t_i$. By definition, we have

$$\alpha \leq \frac{\sum_{i=1}^{n} \alpha_i t_i}{\sum_{i=1}^{n} t_i}$$

□

This tells us that the final α value is essentially a weighted average of the α_i values, weighted by their T-count reductions. It is easy to see that if all decompositions have $\alpha_i \leq \alpha_{max}$, then (3.1) implies the overall $\alpha \leq \alpha_{max}$. The actual α value in practice is likely to be much lower than this upper bound, because of step 3 in Algorithm **??**, where we simplify the diagram using ZX-simplify [9]. In [9], although the BSS decomposition has the $\alpha \approx 0.468$, the actual α value obtained in experiments was lower, at ≈ 0.4 [18].

A step further than using the α value of a decomposition, is to consider the α value when including the simplification step. In [17], this is defined as the *effective* α_e value. If the associated number of T-like spiders removed is r, r_e, where $r \leq r_e$ before and after simplification respectively, then we have

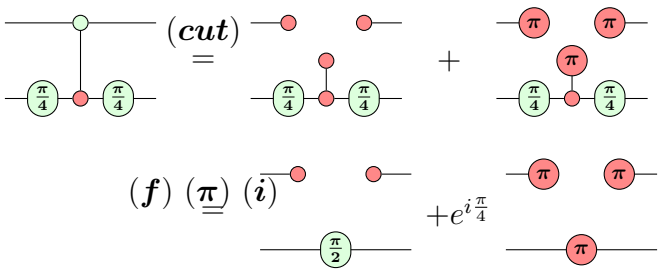
$$\alpha_e = \frac{r}{r_e}\alpha \tag{3.2}$$

This relation can make a significant difference especially for low values of r. In the next section, we also consider the structure of the diagram to increase r_e.

3.3 Graph Structure Heuristic

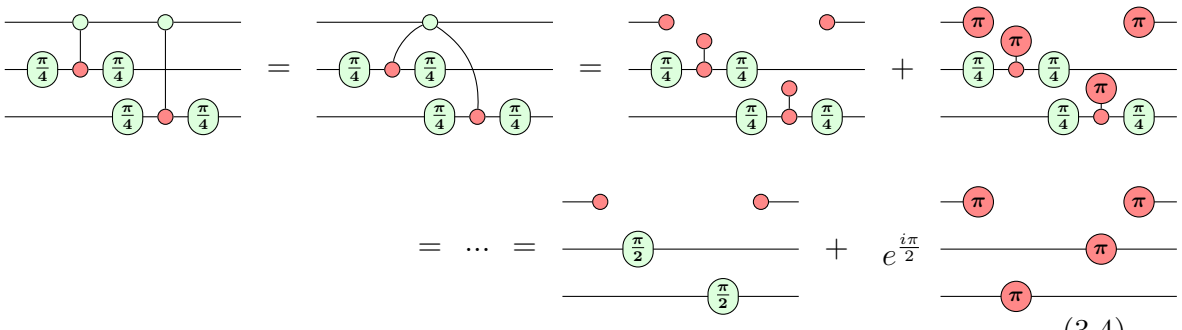
A disadvantage of the greedy algorithm is that it does not take into account the structure of the diagram. When we only consider the α value of the decomposition, we fail to take into account that T-like spiders can be fused in between decompositions where decompositions with seemingly higher α values may actually be more efficient.

In [18], the heuristic used considers the structure of CNOT gates placed in between T gates. The following observation was made.



(3.3)

Using the graph (***cut***), which without simplification has $\alpha = 1$ if the spider being cut is a T-like spider, we can instead obtain a decomposition with $\alpha \approx 0.333$, with simplication, for the reduction of 3 T-like spiders at the cost of 2 terms. In fact, this structure can be stacked in the following way.



(3.4)

Using these observations, a heuristic can be developed, where the main idea is to count the number of T-like spiders that can be removed, divided by how many cuts are needed to remove them.

Definition 3.3.1. Let V be a Clifford+T diagram, and let v be a vertex in V. Let C be the set of sequence of graph cuts. For each $c \in C$ where there are $|c| = n$ cuts

in the sequence, let $t(c)$ be the number of T-like spiders removed by the sequence of cuts using (***cut***), including simplifications between each cut. Then a *cut heuristic* for the sequence c of length n is defined as any heuristic $h : V \to \mathbb{R}^+$ such that

$$\exists c = (v, ..., v_n) \in C, 0 < h(v) \leq \frac{t(c)}{|c|} = h_e(v) \tag{3.5}$$

where $h_e(v)$ is the effective heuristic value of c. ▲

For the cut heuristic to be valid, it should not overestimate the effective heuristic value h_e. For the cut heuristic to be useful, it should be close to the value of h_e. We have that a higher cut heuristic value implies a more efficient decomposition. In fact, the associated α_e value of the sequence of decompositions is given by

$$\alpha_e(c) = \frac{t(c)}{|c|} = \frac{1}{h_e(v)} \leq \frac{1}{h(v)} = \alpha_h(c) \tag{3.6}$$

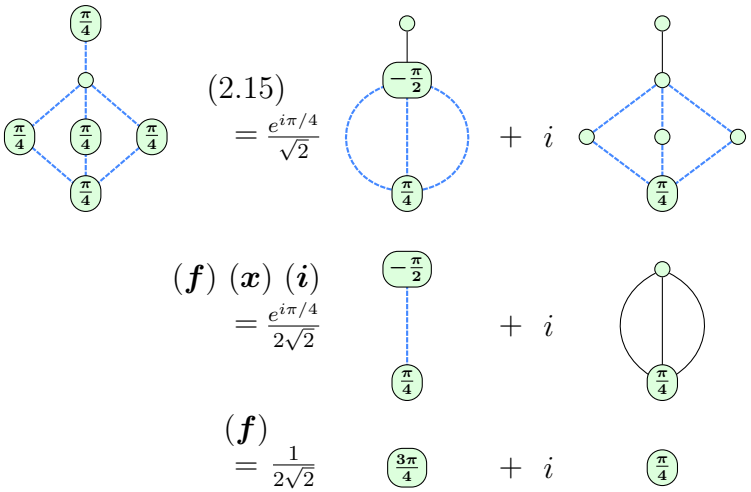
where $\alpha_h(c)$ is the associated α value of the cut heuristic, and since the $n = \log_2(2^n)$ cuts contribute to 2^n terms.

If the sequence of cuts leads to a stabiliser decomposition, the effective cut heuristic is closely related to our α_χ value, but does not take into account the other decompositions we have available. For some Clifford+T diagram V, let C be the set of sequences of decompositions resulting in a stabiliser decompositions that only use graph cuts, and D be the set of stabiliser decompositions. Let $f : C \to D$ be the function that maps a sequence of cuts to the decomposition obtained. Let $c^* = (v^*, ...) \in C$ be the most efficient decomposition using graph cuts such that

$$|f(c^*)| = \min\{|f(c)| \mid c \in C\} \tag{3.7}$$

Here, $\forall c = (v, ...) \in C, h_e(v^*) \geq h_e(v)$. If c^* results in a decomposition where $|f(c^*)| = \chi(V)$, then $\alpha_\chi = \alpha_e(c^*)$ [1]. In general, however, $\alpha_\chi \leq \alpha_e(c^*)$. A simple example of this can be seen in the following diagram, where the decomposition makes use of $|\text{cat}_4\rangle$ (2.15). It starts with a T-count of 5.

[1] Here, and in (3.6), we overload notation by equating $\alpha(f(c^*)) = \alpha(c^*)$



(3.8)

Using only cuts would require 2 of them, which would give 2^2 terms instead of 2 terms. We have

$$\alpha_\chi \leq \frac{1}{5} < \alpha_e(c^*) = \frac{2}{5} \tag{3.9}$$

3.3.1 CNOT Sandwich Heuristic

In [18], the cut heuristic used concentrated on sequences of cuts that were looking to eliminate CNOT gates sandwiched between T-like spiders gates. As a result, 71% of the time, they were able to obtain c^* on small circuits. Ultimately, efficiency values of $0.1 \lesssim \alpha \lesssim 0.2$ were obtained in practice, considering semi-structured diagrams.

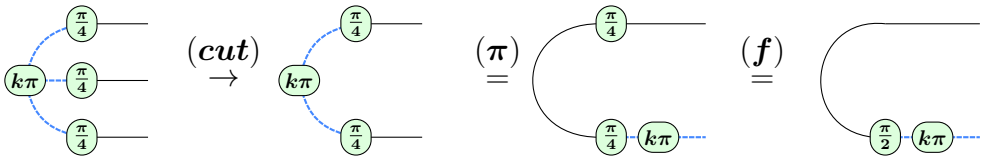
Using the cut heuristic means that there is knowledge on the sequences of cuts to be made. With better consideration of sequences, we can continue to use the greedy approach on α values, with the hope of obtaining a more efficient decomposition. In the tiered approach of [18], a brief description is that increasing lengths of sequences were considered, where each sequence of cuts c_i for tier i led to computation of the cut heuristic $h_{i+1}(v)$, corresponding to sequences $c_{i+1} = k_i : c_i$ for tier $i+1$ for some sequence of vertices k_i. Eventually, the algorithm terminates when there is some $l = i_{\max}$ where there are no sequences of vertices k_l to compute c_{l+1} at tier $l+1$. The result is the computation of h_i values for tiers $i \leq l$, and the choice to cut v where

$$v_{best} := \arg\max_v h_l(v) = \arg\min_v \alpha_{h_l}(v) \tag{3.10}$$

3.3.2 Gadget $|\text{cat}_3\rangle$ Heuristic

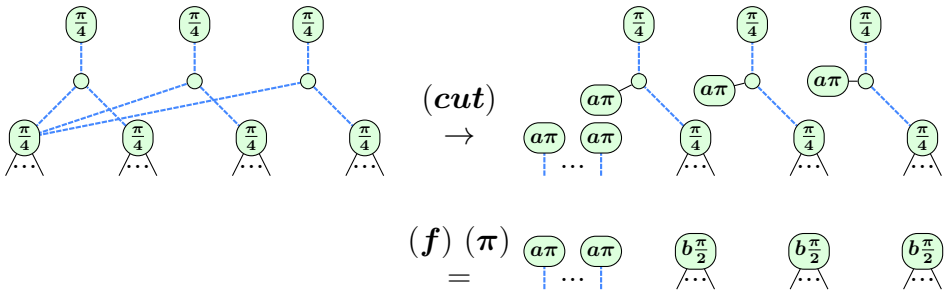
The cut heuristic is also useful outside of CNOT sandwiches. Particularly, the reduced gadget form contains structures that when cut, reduce the T-count greatly.

In [17], the following observation was made when applying the cut to a $|\text{cat}_3\rangle$.



(3.11)

The associated efficiency is $\alpha = 1/3$ here, from removing 3 T-like spiders at the cost of 1 cut. This means that any T-like spider that is part of multiple $|\text{cat}_3\rangle$ can contribute to removing all the associated T-like spiders at once. A visualisation can be seen below (3.12). It can be noted that a $|\text{cat}_3\rangle$ and a two-legged phase gadget are interchangeable concepts in the reduced gadget form.



(3.12)

In [17], the effective α_e value was used, after determining the number of $|\text{cat}_3\rangle$ each T-like spider was part of. Since they were only considering a single cut, the heuristic was essentially equivalent to

$$h(v) = 1 + 2 \cdot (\#\ |\text{cat}_3\rangle\ v \text{ is part of}) \tag{3.13}$$

since each neighbouring $|\text{cat}_3\rangle$ can contribute to the removal 2 T-like spiders.

Proposition 3.3.2. For any ZX-diagram in reduced gadget form, the heuristic $h(v)$ (3.13) is a valid cut heuristic.

$$\exists c = (v, ...) \in C, 0 < h(v) \leq \frac{t(c)}{|c|} = h_e(v)$$

Proof. If v is part of a phase gadget, this is trivial, as $h(v) = 3$. Suppose v has neighbours $w_1, ..., w_k$ that are 0-spiders that are part of $|\text{cat}_3\rangle$ states. Note that $w_1, ..., w_k$ are all part of phase gadgets, by Definition 1.4.5. Each w_i then has 2 legs, one of which is connected to v. Since no two phase gadgets have the same exact legs, we must have for all $w_i, w_j, i \neq j$, that their other legs do not connect to the same vertex u. By cutting only v, we can remove at least $2k + 1$ T-like spiders corresponding to k $|\text{cat}_3\rangle$ states, and vertex v. Then we must have

$$h_e(v) \geq 2k + 1 = h(v) \tag{3.14}$$

□

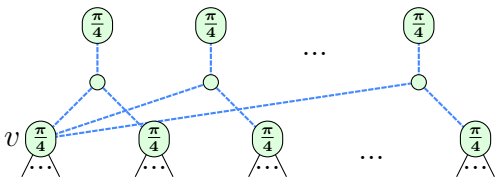


Figure 3.2: Diagram to showcase the heuristic (3.13)

We can note that even if k is small, the heuristic can be quite effective. For instance, with $k = 2, \alpha_h = 0.2$ which is already an improvement over all the $|\text{cat}_n\rangle$ decompositions in [9, 15], the best of which is $\alpha = 0.25$ for the $|\text{cat}_4\rangle$ decomposition. A T-like spider simply has to be connected to 2 $|\text{cat}_3\rangle$ states to improve the decomposition.

More generally, it was also considered that cutting T-like spiders part of $|\text{cat}_n\rangle$ states would convert them to $|\text{cat}_{n-1}\rangle$ states. However, this was not considered in the heuristic of [17], as they focused on obtaining the single-step α_e value.

The heuristic considering $|\text{cat}_n\rangle$ states could then be defined as follows.

$$h(v) = \sum_{i=1}^{k} \frac{2}{n(w_i) - 2} \tag{3.15}$$

where w_i are neighbours of v that are 0-spiders that are part of $\left|\text{cat}_{n(w_i)}\right\rangle$ states, for $n : V \to \mathbb{N}$. The heuristic considers sequences of cuts for every $\left|\text{cat}_{n(w_i)}\right\rangle$ that v is

part of, and attempts to add up each pair of T-like spiders that could be removed with $n(w_i) - 2$ cuts. Unfortunately, this heuristic is not a valid heuristic in general.

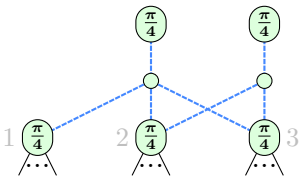


Figure 3.3: Diagram where the heuristic (3.15) is not valid

Here, the heuristic would give $h(v_2) = \frac{2}{1} + 2 = 3$, but cutting 2 vertices to remove 5 T-spiders would mean $h_e(v_2) = \frac{5}{2} < 3$.

We show that with an added condition, the heuristic is valid.

Proposition 3.3.3. Suppose v has 0-spider neighbours $w_1, ..., w_k$ that are part of all $\left|\text{cat}_{n(w_i)}\right\rangle$ states. Let $T(w_i)$ be the set of T-like spiders that is part of each $\left|\text{cat}_{n(w_i)}\right\rangle$ state, so $|T(w_i)| = n(w_i)$. Let $r = \frac{\max\{n(w_i)\}_i - 2}{\min\{n(w_i)\}_i - 2}$. Then for the heuristic $h(v)$ (3.15), we have

$$\left|\bigcup_{i=1}^{k} T(w_i)\right| \geq 2k^2 r \implies \exists c = (v, ...) \in C, 0 < h(v) \leq \frac{t(c)}{|c|} = h_e(v) \tag{3.16}$$

In other words, if the number of distinct T-like spiders part of all the $\left|\text{cat}_{n(w_i)}\right\rangle$ states is at least $2k^2 r$, then the heuristic $h(v)$ is a valid cut heuristic.

Proof. Note that $w_1, ..., w_k$ are all part of phase gadgets, by Definition 1.4.5. By cutting at most $\sum_{i=1}^{k}(n(w_i) - 2)$ T-like spiders as legs of the phase gadgets, we can remove at least $\left|\bigcup_{i=1}^{k} T(w_i)\right|$ T-like spiders. Then we must have

$$h_e(v) \geq \frac{\left|\bigcup_{i=1}^{k} T(w_i)\right|}{\sum_{i=1}^{k}(n(w_i) - 2)} \tag{3.17}$$

Then

$$\begin{aligned} h(v) &= \sum_{i=1}^{k} \frac{2}{n(w_i) - 2} \\ &\leq \sum_{i=1}^{k} \frac{2}{\min\{n(w_i)\}_i - 2} \\ &= \frac{2kr}{\max\{n(w_i)\}_i - 2} \\ &= \frac{2k^2 r}{k(\max\{n(w_i)\}_i - 2)} \\ &\leq \frac{\left|\bigcup_{i=1}^{k} T(w_i)\right|}{\sum_{i=1}^{k}(n(w_i) - 2)} \\ &\leq h_e(v) \end{aligned} \tag{3.18}$$

□

The proposition condition is generally unlikely to be satisfied, and we show in the next chapter that the heuristic can perform poorly for some diagrams. From the proof, we can gather that there is in fact a cut heuristic that is valid.

Corollary 3.3.4. The following heuristic is a valid cut heuristic.

$$h(v) = \frac{\left|\bigcup_{i=1}^{k} T(w_i)\right|}{\sum_{i=1}^{k}(n(w_i) - 2)} \leq h_e(v) \tag{3.19}$$

Unfortunately, although the heuristic is valid, it can vastly underestimate the effective heuristic value. This happens when there is a large overlap in T-like spiders connected to different phase gadgets. A simple example is as follows, where we have $|\text{cat}_n\rangle$ for $n = 3, 4, 5$.

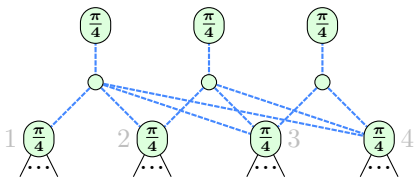
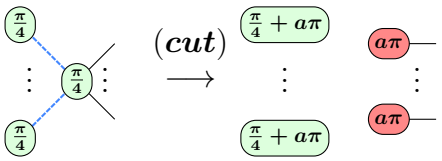


Figure 3.4: Diagram where the heuristic (3.19) underestimates the effective heuristic value

$h_e(v_1) \geq \frac{7}{3}$ since only 3 cuts are needed to remove 7 T-spiders, but $h(v_1) = \frac{4}{3+2+1} = \frac{2}{3} < \frac{7}{3}$.

3.3.3 Lone Phase Heuristic

Another heuristic turns out to be somewhat common to measure is the lone phase heuristic. We can observe the following decomposition.



(3.20)

A T-like spider connected to n magic states  would allow the T-count to drop by n. Here, we can note that the reduction happens even for spiders with odd integer multiples of $\frac{\pi}{4}$. We define the *lone phase heuristic* as follows.

$$h(v) = 1 + \#\ \text{  }\ v \text{ is connected to} \tag{3.21}$$

It's clear to see that this is a valid cut heuristic.

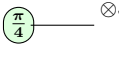
What we have then is that we can combine the two "simple" heuristics (3.13, 3.21) so far. If n is the number of $|\text{cat}_3\rangle$ states v is part of, and m is the number of magic states  v is connected to, then we have the following heuristic.

$$h(v) = 1 + 2n + m \tag{3.22}$$

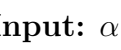
There is clearly no overlap between the two types of T-like spiders that each heuristic considers, so the heuristic is still valid. Surprisingly, just a simple combination of the two heuristics can be quite effective in practice, as we will see in the next chapter.

3.3.4 Greedy Algorithm with Cut Heuristic

A natural extension of the greedy algorithm is to also consider the cut heuristic on top of the cat decompositions.

We have the α values of $|\text{cat}_4\rangle$, $|\text{cat}_6\rangle$, $|\text{cat}_5\rangle$, $|\text{cat}_3\rangle$, and magic state  being $0.25 < 0.264 < 0.317 < 0.333 < 0.396$ respectively. Depending on the value of the best heuristic (3.22), and the best available $|\text{cat}_n\rangle$ decomposition in the diagram, we can continue to use the greedy approach. We have the following simple algorithm.

Algorithm 1 Greedy Algorithm with Heuristic

Input: α values for $|\text{cat}_4\rangle$, $|\text{cat}_6\rangle$, $|\text{cat}_5\rangle$, $|\text{cat}_3\rangle$, and 
Output: Chosen state and corresponding decomposition
Let $\alpha_4 = 0.25, \alpha_6 = 0.264, \alpha_5 = 0.317, \alpha_3 = 0.333, \alpha_{\otimes 5} = 0.396$
Compute the best catstate $|\text{cat}_{best}\rangle$ with the lowest α value
Let α_{best} be the α value of $|\text{cat}_{best}\rangle$
Compute α_h using a heuristic
if $\alpha_h < \alpha_{best}$ **then**
    Use the decomposition corresponding to the heuristic
else
    Use the decomposition corresponding to $|\text{cat}_{best}\rangle$ or 
end if

Even with a small values of n, m, the heuristic can be quite effective. Just having $n = m = 1$ can already give a $\alpha_h = 0.25$, matching the best $|\text{cat}_4\rangle$ decomposition. This heuristic also works without any phase gadgets where $n = 0$, concentrating on the T-like spiders. In this scenario, having $m = 2$ will already give a better $\alpha_h = 1/3$ than the default 5-qubit magic decomposition at $\alpha \approx 0.396$.

3.4 Complexity Analysis

Here, we consider the complexity of computing the heuristics.

Suppose we have a reduced gadget form V with n phase gadgets and m T-like spiders not part of phase gadgets. Let $d(v)$ be the degree of a vertex v.

For the heuristic (3.13), it is easy to compute the heuristic in $O(mn)$ time. Checking which 0-spiders are part of $|\text{cat}_3\rangle$ states can be done in $O(n)$. Each T-like spider v can be processed in $O(d(v)) = O(n)$ time, and checking for all of them takes $O(mn)$, for an overall of $O(n + mn) = O(mn)$ time.

The heuristic (3.15) has a similar complexity of $O(mn)$.

For the $|\text{cat}_n\rangle$ heuristic (3.19), a single T-like vertex v, and its 0-spider neighbours $w_1, ..., w_k$, it takes

$$O\left(d(v) + \sum_{i=1}^{k} d(w_i)\right) = O(n + mn) = O(mn) \tag{3.23}$$

time to compute the heuristic. For all m T-like spiders, this takes $O(m^2 n)$ time.

The lone phase heuristic (3.21) can be computed in $O(m^2)$ time, each T-like spider can be processed in $O(m)$ time, for all m T-like spiders.

Combining the complexities for the general heuristic (3.22) will take $O(mn+m^2)$ time. With the existing greedy algorithm from [9, 15] that has complexity $O(2^{\alpha t}t^2)$, and since $m + n = t$, the complexity remains unchanged. Specifically, looking at the $O(t^2)$ operations in between decompositions,

$$O(t^2 + mn + m^2) = O(t^2) \tag{3.24}$$

Comparing to the method of [17] which requires computing α_e values (3.2) for each of the k decompositions considered to select the best one, the number of operations between each decomposition there is $O(kt^2)$.

Appendices

References

[1] Xiaosi Xu et al. *A Herculean task: Classical simulation of quantum computers*. 2023. arXiv: 2302.08880.

[2] Bob Coecke and Aleks Kissinger. *Picturing Quantum Processes*. Cambridge University Press, 2017. URL: https://books.google.com.sa/books?id=I9gcDgAAQBAJ.

[3] Bob Coecke and Ross Duncan. "Interacting quantum observables: categorical algebra and diagrammatics". In: *New Journal of Physics* 13.4 (Apr. 2011), p. 043016. URL: http://dx.doi.org/10.1088/1367-2630/13/4/043016.

[4] Aleks Kissinger and John van de Wetering. "Universal MBQC with generalised parity-phase interactions and Pauli measurements". In: *Quantum* 3 (Apr. 2019), p. 134. URL: http://dx.doi.org/10.22331/q-2019-04-26-134.

[5] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[6] Aleks Kissinger and John van de Wetering. *Picturing Quantum Software*. 2024.

[7] Ross Duncan et al. "Graph-theoretic Simplification of Quantum Circuits with the ZX-calculus". In: *Quantum* 4 (June 2020), p. 279. URL: https://doi.org/10.22331/q-2020-06-04-279.

[8] Aleks Kissinger and John van de Wetering. "Reducing the number of non-Clifford gates in quantum circuits". In: *Phys. Rev. A* 102 (2 Aug. 2020), p. 022406. URL: https://link.aps.org/doi/10.1103/PhysRevA.102.022406.

[9] Aleks Kissinger and John van de Wetering. "Simulating quantum circuits with ZX-calculus reduced stabiliser decompositions". In: *Quantum Science and Technology* 7.4 (July 2022), p. 044001. URL: http://dx.doi.org/10.1088/2058-9565/ac5d20.

[10] Dave Wecker and Krysta M. Svore. *LIQUi|>: A Software Design Architecture and Domain-Specific Language for Quantum Computing*. 2014. arXiv: 1402.4467 [quant-ph]. URL: https://arxiv.org/abs/1402.4467.

[11] Daniel Gottesman. *The Heisenberg Representation of Quantum Computers*. 1998. arXiv: quant-ph/9807006 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9807006.

[12] Sergey Bravyi, Graeme Smith, and John A. Smolin. "Trading Classical and Quantum Computational Resources". In: *Physical Review X* 6.2 (June 2016). URL: http://dx.doi.org/10.1103/PhysRevX.6.021043.

[13] Sergey Bravyi et al. "Simulation of quantum circuits by low-rank stabilizer decompositions". In: *Quantum* 3 (Sept. 2019), p. 181. URL: http://dx.doi.org/10.22331/q-2019-09-02-181.

[14] Sergey Bravyi and David Gosset. "Improved Classical Simulation of Quantum Circuits Dominated by Clifford Gates". In: *Physical Review Letters* 116.25 (June 2016). URL: http://dx.doi.org/10.1103/PhysRevLett.116.250501.

[15] Aleks Kissinger, John van de Wetering, and Renaud Vilmart. "Classical Simulation of Quantum Circuits with Partial and Graphical Stabiliser Decompositions". en. In: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TQC.2022.5.

[16] Hammam Qassim, Hakop Pashayan, and David Gosset. "Improved upper bounds on the stabilizer rank of magic states". In: *Quantum* 5 (Dec. 2021), p. 606. URL: http://dx.doi.org/10.22331/q-2021-12-20-606.

[17] Julien Codsi. "Cutting-Edge Graphical Stabiliser Decompositions for Classical Simulation of Quantum Circuits". MA thesis. University of Oxford, 2022.

[18] Matthew Sutcliffe and Aleks Kissinger. *Procedurally Optimised ZX-Diagram Cutting for Efficient T-Decomposition in Classical Simulation*. 2024. arXiv: 2403.10964 [quant-ph]. URL: https://arxiv.org/abs/2403.10964.